L07 — Introduction to Sorting; Analysis of Basic Sorting Techniques

Unit: 1 | **Chapter:** Sorting Techniques | **BT Level:** BT4 | **CO:** CO1, CO5

Learning Objectives

- Define sorting and explain its importance in computing
- Classify sorting algorithms by various criteria
- Explain the concept of a stable vs unstable sort
- Describe insertion sort and selection sort with analysis

---

1. What is Sorting?

Sorting is the process of rearranging a collection of elements into a specified order — typically ascending or descending.

Sorting is fundamental because:

- Binary search requires sorted data
- Duplicate detection becomes trivial after sorting
- Many algorithms (interval scheduling, merge-based solutions) assume sorted input
- Database queries, file systems, and search engines all rely on sorting internally

---

2. Classification of Sorting Algorithms

2.1 By Comparison

Type	Description	Examples
Comparison-based	Uses element comparisons	Bubble, Quick, Merge, Heap
Non-comparison	Uses element values/digits	Counting Sort, Radix Sort, Bucket Sort

> **Lower bound theorem:** Any comparison-based sort requires at least Ω(n log n) comparisons.

2.2 By Memory Usage

Type	Description	Examples
In-place	O(1) extra space	Bubble, Selection, Insertion, Quick
Out-of-place	O(n) extra space	Merge Sort

2.3 By Stability

A sort is **stable** if equal elements appear in the same relative order after sorting as before.

Stable	Unstable
Bubble Sort	Quick Sort
Merge Sort	Heap Sort
Insertion Sort	Selection Sort

Why stability matters: When sorting records by multiple keys (e.g., first sort by department, then by name), stability ensures the first sort's order is preserved.

2.4 By Adaptability

Type	Description
Adaptive	Faster on partially sorted input (e.g., Insertion Sort: O(n) for nearly sorted)
Non-adaptive	Same performance regardless of input order (e.g., Selection Sort)

---

3. Selection Sort

3.1 Algorithm

In each pass, find the **minimum element** from the unsorted portion and swap it into its correct position.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):          # i: boundary of sorted portion
        min_idx = i
        for j in range(i + 1, n):   # Find min in unsorted portion
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  # Swap
    return arr
```

3.2 Trace

Array: `[64, 25, 12, 22, 11]`

Pass	Unsorted Portion	Min Found	After Swap
i = 0	[64, 25, 12, 22, 11]	11 at idx 4	**[11**, 25, 12, 22, 64]
i = 1	[25, 12, 22, 64]	12 at idx 2	[11, **12**, 25, 22, 64]
i = 2	[25, 22, 64]	22 at idx 3	[11, 12, **22**, 25, 64]
i = 3	[25, 64]	25 at idx 3	[11, 12, 22, **25**, 64]

Result: `[11, 12, 22, 25, 64]`

3.3 Complexity

Case	Comparisons	Swaps	Time
Best	n(n-1)/2	n-1	$O(n^2)$
Worst	n(n-1)/2	n-1	$O(n^2)$
Average	n(n-1)/2	O(n)	$O(n^2)$

- **Space:** O(1) — in-place
- **Stable:** No (a distant swap can change relative order of equal elements)
- **Adaptive:** No (always does the same number of comparisons)

3.4 When to Use

- When **write operations are expensive** — selection sort makes at most n-1 swaps (vs insertion sort's $O(n^2)$ shifts)
- For **small arrays** or **hardware with slow writes**

---

4. Insertion Sort

4.1 Algorithm

Build a sorted sequence one element at a time. For each new element, insert it into its correct position within the already-sorted portion.

```
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]        # Element to be inserted
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]  # Shift right
            j -= 1
        arr[j + 1] = key         # Insert in correct position
    return arr
```

4.2 Trace

Array: `[5, 2, 4, 6, 1, 3]`

i	key	Sorted portion before	After insertion
1	2	[5]	[**2**, 5]
2	4	[2, 5]	[2, **4**, 5]
3	6	[2, 4, 5]	[2, 4, 5, **6**]
4	1	[2, 4, 5, 6]	[**1**, 2, 4, 5, 6]
5	3	[1, 2, 4, 5, 6]	[1, 2, **3**, 4, 5, 6]

Result: `[1, 2, 3, 4, 5, 6]`

4.3 Complexity

Case	Comparisons	Shifts	Time
Best (sorted)	n-1	0	**O(n)**
Worst (reverse sorted)	n(n-1)/2	n(n-1)/2	$O(n^2)$
Average	$n^2/4$	$n^2/4$	$O(n^2)$

- **Space:** O(1) — in-place
- **Stable:** Yes
- **Adaptive:** Yes — performs O(n) for nearly sorted arrays

---

5. Comparison of Basic Sorts

Algorithm	Best	Worst	Average	Space	Stable
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Bubble Sort (next)	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes

---

6. Overview of All Sorting Algorithms (Preview)

Algorithm	Best	Average	Worst	Space	Notes
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Simple, stable
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Min swaps
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Best for small/nearly sorted
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Stable, divide & conquer
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	Fast in practice
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	In-place, not stable
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(k)	Non-comparison
Radix Sort	O(d(n + k))	O(d(n + k))	O(d(n + k))	O(n + k)	Non-comparison

---

Summary

- Sorting is foundational to algorithms and data processing.
- Algorithms are classified by: comparison vs non-comparison, in-place vs out-of-place, stable vs unstable, adaptive vs non-adaptive.
- **Selection sort:** Always $O(n^2)$, makes fewest swaps.
- **Insertion sort:** O(n) for sorted input, $O(n^2)$ worst case — best simple sort for nearly sorted data.
- The optimal comparison-based sort lower bound is Ω(n log n).

References

- Cormen et al., *Introduction to Algorithms*, Ch. 2 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 4 (Springer, 2020)

- Laaksonen, *Guide to Competitive Programming*, Ch. 3 (Springer, 2024)